GOLD: Definite Loops (Fri Sep 11)

Increment/Decrement Operators

Post- and Pre- Increment/Decrement

- `x++` increments x by one, but the expression value is the original x
  - `int x = 0;`
  - `IO.println(x++);`
  - `IO.println(x);`
- Prints 0, then 1

- `++x` increments x by one, and its value is the new x
  - `int x = 0;`
  - `IO.println(++x);`
  - `IO.println(x);`
- Prints 1, then 1

- `x++` and `x--` are very common idioms

- Life becomes messy if an expression contains multiple pre- and post- increment and decrement operators

From While to For

Problem: Print “hello” 10 times using a while loop

We illustrate using a while loop to implement a definite iteration

```
void main() {
    int i = 0;
    while (i < 10) {
        IO.println(String.format("Hello (#%d)", i));
        i++; // increment i by 1, but return old i
    }
}
```

There is another way we could do this:

```
void main() {
    /* Equivalent to:
    int i = 0;
    while (i < 10) {
        System.out.printf("Hello (#%d)\n", i);
        i++;
    }
    */
    for (int i = 0; i < 10; i++) {
        IO.println(String.format("Hello (#%d)", i));
    }
}
```

The For Loop

From `while` to `for`

A counted `while` loop has its three jobs scattered across three lines:

```
void main() {
    int i = 0;          // 1. initialize
    while (i < 3) {     // 2. test
        IO.println("hello");
        i++;            // 3. update
    }
}
```

The `for` loop gathers all three onto **one line**:

```
void main() {
    for (int i = 0; i < 3; i++) {
        IO.println("hello");
    }
}
```

We have the same three jobs

1. **initialize**
2. **test**
3. **update**

just packed together where you can see them at a glance.

The `for` loop

```
for (initialize; test; update) {
    // body
}
```

Java runs it in this order:

1. **initialize** once, at the start
2. **test:** if false, stop; if true, run the body
3. **update** and then go back to **step 2**

```
for (int i = 1; i <= 5; i++)
    IO.println(i);          // prints 1 2 3 4 5
```

All the details

The **initialize** part runs once, at the start, and can be:

- omitted;
- a declaration and initialization of a loop-local variable (e.g. `int i = 0`);
- several declarations of the **same type**, separated by commas (e.g. `int i = 0, j = 9`).

The **test** is a boolean expression evaluated **before every pass**:

- if false: the loop ends, skipping to the statement after the body;
- it can be omitted too: then it counts as `true` (an infinite loop unless something inside `break`s out).

The **update** runs **after every pass** and can be:

- omitted;
- one expression (e.g. `i++`);
- several expressions separated by commas (e.g. `i++, j--`).

A variable declared in **initialize** exists only inside the loop: after the closing brace, `i` is gone.

Common For Loops

The most common idiom

To do something **n times**, count from `0` up to `n - 1`:

```
for (int i = 0; i < n; i++) {
    // runs exactly n times
}
```

Why `0` and `< n` (not `1` and `<= n`)?

Because **strings and arrays (lists) number their positions from 0** . `i` then ranges over each valid position.

Use `< n`; using `<= n` runs one time too many (the "fencepost" bug).

Looping over a String

A String numbers its characters from `0` to `length() - 1`. A `for` loop visits each one:

```
void main() {
    String s = "hello there world";
    for (int i = 0; i < s.length(); i++)
        IO.println(String.format(
            "s.charAt(%d) = '%c'",
            i, s.charAt(i)));
}
```

Pause and predict

```
for (int i = 0; i <= 5; i++) {
    IO.println(i);
}
```

How many times does the body run?

- **A.** 5 times

- **B.** 6 times
- **C.** 4 times
- **D.** It never stops

Problem: Count the Vowels

Problem: Let's count the number of vowels in a String.

A String numbers its characters from `0` to `length() - 1`. A `for` loop visits each one:

```
void main() {
    String word = IO.readln("A word? ");

    String w = word.toLowerCase();     // so 'A' and 'E' count too
    int vowels = 0;
    for (int i = 0; i < w.length(); i++) {
        char c = w.charAt(i);
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
            vowels++;
        }
    }
    IO.println(String.format("%s has %d vowels", word, vowels));
}
```

Desired output:

```
A word? banana
banana has 3 vowels
```

Problem: Palindrome Again

Problem: Implement a method in class `Palindrome`

`boolean isPalindrome(String s)`

to test if s is a palindrome, a string that reads the same backwards as forwards.

Wednesday's Approach:

- We used a while loop `while (s.length() > 1)`
- Each step compared the first and last characters:
  `if (s.charAt(0) != s.charAt(s.length() - 1))`
- The update created a new String each time: `s = s.substring(1, s.length() - 1)`
- This can be slow if the String is huge

Today's Approach: Use a `for` loop and a more efficient approach

- Iterate through the first half of the string
- Compare current character to corresponding character at other end of string
- How do we find the right index to compare to?
  - Consider a table of indexes `i` and `j` for "racecar"
  - Find the pattern, generalize to solution

```
boolean isPalindrome(String s) {
    if (s == null)
        return true;

    for (int i = 0; i < s.length() / 2; i++) {
        if (s.charAt(i) != s.charAt(s.length() - i - 1))
            return false;
    }
    return true;
}
```

▶ Expand

Pause and predict

```
int x = 12;
int y = 18;
int i;
for (i = 1; i <= y && i <= x; i++) {
    if (x % i == 0 && y % i == 0) {
        IO.println(i);
        continue;
    }
}
IO.println("All done at " + i);
```

What is the **last** number printed before `All done`?

- **A.** `1`
- **B.** `3`
- **C.** `6`
- **D.** `13`

do-while

`do-while`: run the body at least once

A `while` loop checks its condition *before* the first pass, so it can run **zero** times.

Sometimes the body must run **once before there is anything to test**:

```
do {
    statements;
} while (boolean-expression);
```

1. **Execute** the statements in the body
2. **Test** the boolean expression
3. If true, **repeat** from step 1



The semicolon after `while (…)` is **required** here.

The classic use: input validation

Ask for input and keep asking until the answer is acceptable.

You can't test an answer you haven't asked for yet, so the loop body must run first:

```
void main() {
    int n;
    do {
        n = Integer.parseInt(IO.readln("Pick a number from 1 to 10: "));
    } while (n < 1 || n > 10);
    IO.println("Thanks: " + n);
}
```

Which loop?

We've now seen three shapes for a sentinel loop:

1. ask before *and* inside the loop (here);
2. `while (true)` with a `break` (here);

3. or the `readln` tucked into the condition (too clever):

```
void main() {
    int sum = 0;
    int count = 0;

    String response;
    while (! (response = IO.readln("Enter a number (or \"done\"): ")).equals("done"))
        int num = Integer.parseInt(response);
        // running total: sum
        sum = sum + num;
        // running count
        count = count + 1;
    }
    IO.println(String.format("The total is %d from %d numbers entered.",sum,count));
}
```

⚠️

The semicolon after `while (…)` is **required** here.

`do-while` does **not** fit Summer well.

To run the loop body before taking input or checking for "done," you would have to pretend a first number of 0 was entered.

Use `do-while` in those cases when the body must run first: validation and menus (the end of today).

Pause and predict (poll)

```
int i = 5;
do {
    IO.println(i);
    i++;
} while (i < 0);
```

What prints?

- **A.** Nothing. The test is false from the start
- **B.** `5`, once
- **C.** `5 6 7 8 …` forever
- **D.** Compile error

Museum of Bugs

Museum of Bugs

- The infinite loop

```
void main() {
    int n = 1;
    while (n < 100) {
        IO.println(String.format("n = %d", n));
        // forgot to increment n here
    }
}
```

- The *almost* infinite loop

```
void main() {
    // count down to blast off
    for (int i = 10; i > 0; i++)
        IO.println(String.format("%d", i));
    IO.println("BLAST OFF!");
}
```

    - Why is this not an actually infinite loop?
    - What will happen?

- Fencepost error

```
void main() {
    for (int i = 0; i <= 5; i++)
        IO.println(String.format(
        "print this line 5 times (%d)", i));
}
```

- The loop that never looped

```
void main() {
    int number = 0;
    int sum = 0;
    // read ints from user until zero, then print sum
    while (number > 0) {
        sum += number;
        number = Integer.parseInt(IO.readln("Enter postive int (0 to quit): "));
    }
    IO.println(String.format("sum = %d", sum));
}
```

- Another skipped loop

```
void main() {
    int i = 10;
    while (--i >= 0); {
        IO.println(String.format("message #%d", i));
    }
}
```

Watch out for those semicolons!

Except in `do-while` which does need one!

```
do {
    something;
} while (i > 0);
```

Pause and predict

Q. What happens here?

```
for (int i = 0; i < 3; i++); {
    IO.println("hi " + i);
}
```

- **A.** Prints `hi 0`, `hi 1`, `hi 2`
- **B.** Prints `hi 3`, once
- **C.** Compile error: `i` cannot be found
- **D.** Prints nothing at all

Nested Loops

Just like you can nest if statements, you can also nest loops.

The inner loop is run completely for each iteration (each pass) of the outer loop:

```
void main() {
    for (int i = 0; i < 5; i++)
        for (int j = 0; j < 5; j++)
            IO.println(String.format("i = %d, j = %d", i, j));
}
```

We can make a multiplication table, but we need the columns to line up.

Here's a new format specified: `%3d` means print an integer in this hole, and make it **at least** `3` places wide.

If the number has more than 3 digits, that's OK.

If the number has less than 3 digits, we pad with spaces on the **left.**

```
void main() {
    for (int i = 1; i <= 3; i++) {
        String row = "";
        for (int j = 1; j <= 3; j++)
            row += String.format("%3d", i * j);
        IO.println(row);
    }
}
```

Loops Summary

The three loops

Loop	Use it when…	Shape
`for`	you're counting a known number of times	`for (int i = 0; i < n; i++) { … }`
`while`	you repeat until some condition, count unknown	`while (notDone) { … }`
`do-while`	same, but the body must run **at least once**	`do { … } while (notDone);`

All three perform the same three jobs: initialize, test, update.

Pick the one that reads most clearly.

Interactive Menus

Interactive menus: keep asking until "quit"

Lots of programs don't run once and stop.

Menus keep offering choices until you're done.

Think of a game menu, an ATM, or an ordering kiosk.

That pattern combines two tools you already have:

- a **`do-while`** loop : show the menu **at least once**, and repeat until the user quits
- an **`arrow switch`** : run the right action for whatever they chose.

The menu-loop pattern

```
String choice;
do {
    // 1. print the menu of options
    choice = IO.readln("Choose: ");
    // 2. act on the choice
    switch (choice) {
        case "1" -> IO.println("option 1 chosen");
        case "2" -> IO.println("option 2 chosen");
        case "q" -> IO.println("Goodbye!");
        default  -> IO.println("Not a valid choice.");
    }
} while (!choice.equals("q"));
```

- **`do-while`** shows the menu before getting/checking input
  - the user is prompted at least once
- **`switch`** sends each choice to its own action.
  - No need for parsing or `equals`
- **`default`** catches typos instead of crashing or ignoring them

- The loop ends when the choice is the sentinel

Extra Reading: Divisor Pattern

Extra Reading: Divisor Pattern

This slide is extra reading that you can read at home to get extra practice with nested loops.

Print an NxN table where the entry at row i and column j has an * if i divides j or j divides i

Example for N == 5:

```
    1  2  3  4  5
1   *  *  *  *  *
2   *  *     *
3   *     *
4   *  *     *
5   *           *
```

In the formatting above, we have two spaces between every column.

Let's try it:

```
void main() {
    String response = IO.readln("Enter a positive integer: ");
    int N = Integer.parseInt(response);

    IO.print("   ");
    for (int j=1; j<=N; j++)
        IO.print(String.format("%3d", j));
    IO.println();

    for (int i=1; i<=N; i++) {
        IO.print(String.format("%3d", i));
        for (int j=1; j <= N; j++) {
            if (i % j == 0 || j % i == 0)
                IO.print("  *");
            else
                IO.print("   ");
        }
        IO.println();
    }

}
```

Extra Reading: The Boiler Burgers Kiosk

Build it: the Boiler Burgers kiosk

Extra reading (not covered in lecture).

This is the complete worked example of the menu-loop pattern from the Interactive Menus slide.

Read it at home, run it, and try adding a fifth option (say, "Remove a burger" or "remove all items").

Here is a complete text-based kiosk that lets the user build an order through choosing options "Add a burger" or "Add fries".

We should be able to see the cost so far with "Show total" and then "Checkout"

```
void main() {
    final double BURGER_PRICE = 5.00;
    final double FRIES_PRICE = 3.00;

    String choice;
    int burgers = 0, fries = 0;
    do {
        // 1. print the menu of options
        IO.println("== Boiler Burgers ==");
        IO.println("1) Add a burger");
        IO.println("2) Add a fries");
        IO.println("3) See total");
        IO.println("4) Checkout and pay");
        choice = IO.readln("Choose: ");
        // 2. act on the choice
        switch (choice) {
            case "1" -> burgers++ ;
            case "2" -> fries++ ;
            case "3" -> {
                double total = burgers * BURGER_PRICE + fries * FRIES_PRICE;
                IO.println(String.format("Your total is %.2f",total)) ;
            }
            case "4" -> {
                double total = burgers * BURGER_PRICE + fries * FRIES_PRICE;
                IO.println(String.format("Burgers: %d at %.2f each", burgers, BURGER_P
                IO.println(String.format("Fries:   %d at %.2f each", fries, FRIES_PRIC
                IO.println(String.format("Your total is %.2f",total));
            }
            default  -> IO.println("Not a valid choice.");
        }
    } while (!choice.equals("4"));

}
```

Take Home: Reverse a String

Implement a method

`void printReverse(String s)`

that prints `s` in reverse **on one line**, character by character, followed by a newline.

```
Enter a string: hello
olleh
```

Reminder: `IO.print(x)` prints `x` **without** starting a new line afterwards. Using this method a loop can build up a line one character at a time. Finish with `IO.println()` (no arguments) to end the line.

A `for` loop that counts **down** from `s.length() - 1` to `0` is the shape you want.

Press **Test** to run the tests.